

Security incident report

Section 1: Identify the network protocol involved in the incident

The protocol involved in this incident is HTTP (Hypertext Transfer Protocol). Since the issue centred on accessing the web server for `yummyrecipesforme.com`, requests to load a webpage are inherently HTTP traffic. This was confirmed when `tcpdump` was run and the site was accessed — the resulting log showed HTTP being used to contact the server. The malicious file itself was also delivered to users' computers via HTTP at the application layer, meaning the same protocol used to legitimately load the page was also the vehicle for the malware download.

Section 2: Document the incident

Several customers contacted the website's helpdesk to report that, upon visiting the site, they were prompted to download and run a file said to unlock access to new recipes. Since running that file, their computers have been operating slowly. Separately, the website owner attempted to log into the web server's admin account and found they had been locked out.

To investigate without risking the company network, the analyst set up a sandbox environment and ran tcpdump to capture traffic while interacting with the site. On visiting the page, the analyst was prompted to download a file claiming to offer free recipes, accepted it, and ran it — after which the browser redirected to a fake domain, greatrecipesforme.com.

Reviewing the tcpdump log, the analyst observed that the browser first requested the IP address for yummyrecipesforme.com and established a connection over HTTP, at which point the file was downloaded and executed. Shortly after, the log showed a sudden shift in traffic as the browser issued a new DNS request, this time for greatrecipesforme.com, with traffic then rerouted to that new IP address.

A senior analyst subsequently reviewed the source code for the site and the downloaded file, and found that the site had been altered to insert code prompting visitors to download a malicious file disguised as a browser update. Combined with the website owner's report of being locked out of the admin account, the team's assessment is that the attacker used a brute force attack to gain access to the admin panel and change the password, locking out the legitimate owner. Running the malicious file is what ultimately compromised end users' computers.

Section 3: Recommend one remediation for brute force attacks

The primary security measure recommended to prevent future brute force attacks is disallowing the reuse of previous passwords. Since the vulnerability that enabled this attack was the continued use of a default password, preventing old or default passwords from being reused during a reset directly closes that gap. Alongside this, requiring more frequent password changes would help — if a password is ever compromised without the team's knowledge, updating it regularly reduces the window in which it remains useful to an attacker. Finally, implementing two-factor authentication (2FA) would add a further layer of protection: even if login credentials are guessed, the attacker would still need to confirm a one-time passcode sent to the account holder's email or phone before gaining access. Between these three measures, a brute force attack becomes significantly less likely to succeed even if a password is compromised.

